

Overview

How to Prepare 🎧

Practice strategies, environment setup, and what to do the day before your AI-enabled coding interview.

Try AI-Coding Yourself

Practice real problems in an AI-enabled environment

 Open Sandbox

Preparing for an AI-enabled coding interview is different from grinding leetcode, but it's not a completely different world. A lot of the fundamentals carry over. The difference is in what you emphasize and what you practice.

1. **Algorithms still matter** - but for recognition and verification, not memorization
2. **Reading unfamiliar code** - the most undertrained skill from traditional prep
3. **Coding with AI under time pressure** - the single most important practice activity
4. **Format-specific prep** - tailored to structured vs. open-ended

Algorithms still matter

Your leetcode practice isn't wasted. But shift your focus away from implementation speed and toward something more useful for this format.

In an AI-enabled interview, the AI can implement BFS, write a merge sort, or build a trie from scratch. What it can't reliably do is pick the right algorithm for your specific constraints and codebase context. That judgment call is still on you.

The skill that matters now is recognition. When you see a problem involving shortest paths, you need to think "this is a BFS/Dijkstra problem" quickly and confidently. When you see overlapping subproblems, you need to recognize that dynamic programming applies. When the problem requires efficient lookups, you need to know that a hashmap is the right tool. The AI will write the implementation. You need to pick the right one and know enough to verify it's correct.

This means your practice should shift toward recognition speed. Read a problem's first paragraph, stop, and write down the data structure and the pattern. Shortest path → BFS/Dijkstra. Efficient lookup → hashmap. Overlapping subproblems → DP. Then check your answer against the full solution. Practice explaining *why* you'd choose a particular approach — that reasoning is what interviewers are listening for when you narrate your thought process.



A good exercise — go through leetcode problems and just classify them without solving them. Read the problem, identify the approach (sliding window, BFS, union-find, etc.), and move on. Speed of recognition matters more than speed of implementation when you have AI doing the typing. If you want to build the underlying intuition, our [DSA learning path](#) teaches core patterns with interactive visualizations.

You also need to know enough about algorithms to spot when the AI gets it wrong. If you ask for a DFS that handles cycles and the AI returns one that doesn't track visited nodes, you need to catch that. If you ask for an efficient graph traversal and the AI gives you $O(V^2)$ instead of $O((V+E) \log V)$, you need to notice. The bar has shifted from "can you write it" to "do you understand it well enough to direct and verify."

Practice reading unfamiliar code

In traditional prep, you always start with a blank file. In AI-enabled interviews, it's very common to start with hundreds of lines of code you've never seen before. We cover the specific techniques for navigating unfamiliar code in [codebase orientation](#).

Find open-source projects on GitHub and practice orienting yourself quickly. Give yourself 5 minutes per project. A few good ones to try: [expressjs/express](#) (Node.js, well-structured), [fastapi/fastapi](#) (Python API, clean architecture), [redis/redis](#) (C, complex, harder). Where's the entry point? What are the main classes? How does data flow? Use AI to help summarize files and explain patterns, then verify its answers by actually reading the code.

Tools like [Google Code Wiki](#) and [DeepWiki](#) are useful for practice. They let you explore open-source codebases interactively, so you can form your own understanding and then check it against what the tool says. In the actual interview you won't have these. The point is to build the habit, not rely on the tool.

Practice coding with AI under time pressure

Set a timer for 45-60 minutes and work through a multi-step problem with your AI tools. This is the most important prep activity you can do, and most candidates don't do nearly enough of it.

The problem should be big enough that you couldn't solve it by hand in the time limit. Ideally it has multiple phases, like building a feature, then adding error handling, then optimizing for performance. You can find problems like this on any coding challenge site, just pick medium-to-hard problems and combine two or three into a single session.



Our [community questions database](#) has recently asked AI coding interview questions submitted by real candidates. Try working through them in our [AI-Coding Sandbox](#) to simulate the full interview experience under realistic conditions.

Practice the full cycle every time. Start by reading the problem and forming a plan, then state that plan out loud even if nobody is listening. From there, direct the AI to implement it piece by piece, reviewing what it generates and running the code before moving on. When something breaks, fix it and keep going. The point is to build the rhythm so that on interview day, juggling all of these things feels natural rather than overwhelming.

Record yourself if you can. When you watch it back, look for the three failure modes that show up over and over in candidate feedback: going silent for too long, accepting AI output without really checking it, and losing time because you didn't have a plan going in. All three are fixable with practice, but you have to see yourself doing them first.



The biggest mistake candidates make is practicing with AI in a relaxed, untimed setting and then being surprised by how different it feels under pressure. Time pressure changes everything. Your prompts get sloppier, you start skipping verification, and you stop narrating. Practice under realistic conditions.

Format-specific prep

Beyond the universal skills above, tailor your prep to the [specific format](#) you'll be facing.

Structured prep

If the company offers a practice environment, do a real test drive. Don't just open the IDE and type a few lines. Work through an actual multi-phase problem end to end. Pay attention to how fast the AI responds (often slower than what you're used to), which models are available, and how multi-file navigation works. Candidates who show up having never used the platform waste valuable minutes on mechanics. Our [AI-Coding Sandbox](#) simulates the structured interview experience with real multi-phase problems and an AI assistant, so you can build the muscle memory before your actual interview.

Practice working with a limited AI. The platform AI is going to be more constrained than your local setup, and some candidates report it being significantly less helpful than their normal tools. Practice solving problems where you lean on the AI for boilerplate and simple implementations but handle the hard thinking yourself. If the AI can only get you 60% of the way there, can you comfortably cover the remaining 40%?

Open-Ended prep

Use whatever you're already using day-to-day. Don't switch tools for the sake of the interview. Familiarity matters more than using the "best" tool.

If you're not currently using an AI-native editor, now is the time to set one up. [Cursor](#) and [Claude Code](#) are the most popular choices. Spend a few sessions getting comfortable before the interview so you're not learning the interface under pressure.

Set up your IDE exactly how you plan to use it in the interview. Make sure screen sharing works and that everything the interviewer needs to see is visible, including your AI interactions. Some screen sharing setups crop out sidebars or overlay panels. Test this with a friend or a recording before the interview.



The AI tooling space moves fast. Showing up with state-of-the-art tools and knowing how to use them well signals to interviewers that you're current with how engineering actually works today. Companies running open-ended interviews are specifically looking for candidates who have embraced AI-assisted development, not just dabbled in it.